DeepEP-Ascend × DeepGEMM-Ascend 技术解读

从 Atlas 950 SuperPoD Flex 组网到 MoE 通信与 GEMM 算子的核心算法实现

时间：2026-10



https://gitcode.com/cann

CANN

目录

Part 1 Ascend 硬件平台

Part 2 DeepEP-Ascend 仓库

Part 3 DeepGEMM-Ascend 仓库

硬件：SuperPoD Flex 组网 · 950DT AICore 与存储层级
DeepEP：概览 · asc-comm API · Dispatch / Combine 算子 · 性能
DeepGEMM：概览 · 稠密 GEMM · MegaMoE · 性能

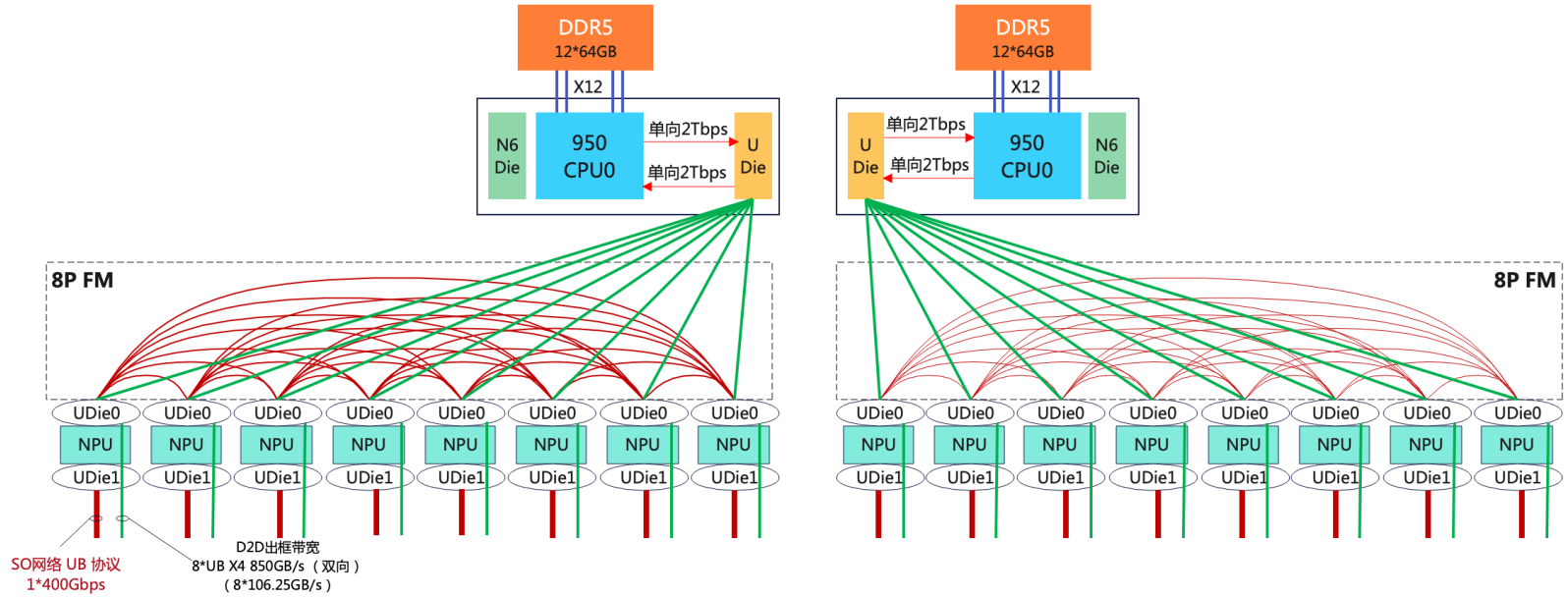
01

Ascend 硬件平台

Atlas 950 SuperPoD Flex 的组网结构（PC16 机型 / UBL128）

Ascend 950DT：AICore 结构 · HBM 带宽 · L2 / L1 / L0 / UB 规格

Atlas 950 SuperPoD Flex：组网结构（PC16 机型）



8P FM ×2：每 FM 8 张 950DT，UDie0/UDie1 双 UB Die 全互联；950 CPU0 + DDR5 12×64GB 挂 U Die

NPU UB port map

Port	0	1	2	3	4	5	6	7	8
UDie1		8 口 for 128 超节点 (scale-up)							
Port	1口H2D	7 口 8 卡 FM							Scale-out

CPU UB port map

Port	0	1	2	3	4	5	6	7	8
UDie	1 口 H2D	1 口 H2D	1 口 H2D	1 口 H2D	1 口 H2D	1 口 H2D	1 口 H2D	1 口 H2D	
	NPU1	NPU2	NPU3	NPU4	NPU5	NPU6	NPU7	NPU8	

UBL128 组网能力

128 卡 3.2 Tbps 单层交换网络（scale-up）

256K 卡 Scale-Out 网络

支撑超低时延推理与前沿基座模型的大模型训练需求

链路带宽

D2D 出框：8×UB X4，**850 GB/s 双向**（8×106.25 GB/s）

SO 网络：UB 协议 **1×400 Gbps**

CPU 侧：950 CPU0 ↔ U Die 单向 **2 Tbps**

UB 端口分配

NPU：UDie1 8 口 for 128 超节点；UDie0 为 1 口 H2D + 7 口 8 卡 FM

CPU：8 口各 1 口 H2D，分别对 NPU1–8

Ascend 950DT：AICore 结构与存储层级

AICore：AIC + AIV 的双核结构

- 单芯 **32 个 AIC（Cube 核）**：矩阵乘 MAD 指令，K 维必须 64 对齐
- **64 个 AIV（Vector 核）**：每个 AIC 带 2 个，负责向量计算、搬运、规约、量化
- 两类核物理分离，协作只能经 **L1 / UB + 事件**，无共享寄存器——这是后续所有 kernel 结构的根源
- 时钟 1.65 GHz

存储层级（单核视角）

HBM（GM） 带宽 **3500 GB/s**，跨 rank 经 URMA 直访

L2 Cache **128 MB** · 4500 GB/s · sector 128 B

L1 512 KB（AIC 侧，GM→L1 顺带 nd2nz）
UB 256 KB/核（AIV 侧）

L0A / L0B 各 64 KB · **L0C 256 KB**（FP32 累加器）· **L0_MX** 缩放因子通路

对两个库设计的影响

硬件特性	设计后果
AIC / AIV 分离	kernel 写成 __mix__ 混合：Cube 算 MAD，Vector 做搬运/规约/量化
L0C 仅 256 KB	BLOCK_M×BLOCK_N 上限直接被它约束
NZ 分形布局	FRAC_MN=16、分形行 32 B；SF 要求 MN-major 打包 int16
URMA / 对称内存	DeepEP 整库建立在 URMA 单写语义上

参数来源：DeepGEMM-Ascend Heuristics 的 HWDesc 硬件描述表与两库 README。L1/L0/UB 容量、MAD_K=64、FRAC_MN=16 等常量直接决定后文所有算子的分块与流水策略。

02

DeepEP-Ascend 仓库

总体功能概览 · asc-comm 通信 API 能力

Dispatch 算子（整体结构 + 核心算法）· Combine 算子 · EPBuffer 性能设计 · 性能解读

总体功能概览：四大 Buffer，API 对齐 DeepEP

Buffer	场景	能力要点
EPBuffer	MoE 专家并行 dispatch / combine（all-to-all）	FP8 量化 dispatch；defer epilogue 与计算重叠；cached mode 复用槽位（解码低延迟）
PPBuffer	流水并行（PP）层间 send / recv	点对点 URMA 直写，绕开集合通信开销
BucketBuffer	CP / DP 集合通信	batched all-gather，单次最多 64 个张量
EngramBuffer	远端内存表读取	按索引从对端对称内存 fetch 条目

与 DeepEP 的关系

- Python API 对齐 **EPBuffer V2.5**：同名方法、同签名，上层框架（vLLM / Megatron 等）可低成本迁移
- 通信全程设备内完成：kernel 直接组装 SQE、写 SQ、敲门铃，Host 只做一次性建组与内存注册
- 当前限制：do_expand=True 必须、do_handle_copy=False、topk_idx 固定 int64；暂不支持混合通信、CPU Engram、graph capture

asc-comm 通信 API 能力：DeepEP 脚下的公共底座

ASC-COMM 是 CANN 的高性能自定义通信编程库：**Host 控制面**（通信域、拓扑、内存注册、Channel、远端地址准备）与 **Device 数据面**（任务构造、传输、归约、同步）分离，资源在 kernel 外准备一次。

三层编程界面（由抽象到底层）

API 层	内存模型	能力
AIN	对称内存	按目标成员 + 窗口 + 偏移发起 Put / Get / Signal，简化集群单边编程
Hcomm 层	非对称内存	Channel 确定通信对端，本地/远端地址分别指定；适合目的地址动态变化场景
Jetty 层	非对称内存	URMA 底层原语（SQE / doorbell / CQE），最灵活、性能最高

三种通信引擎 × 两种协议

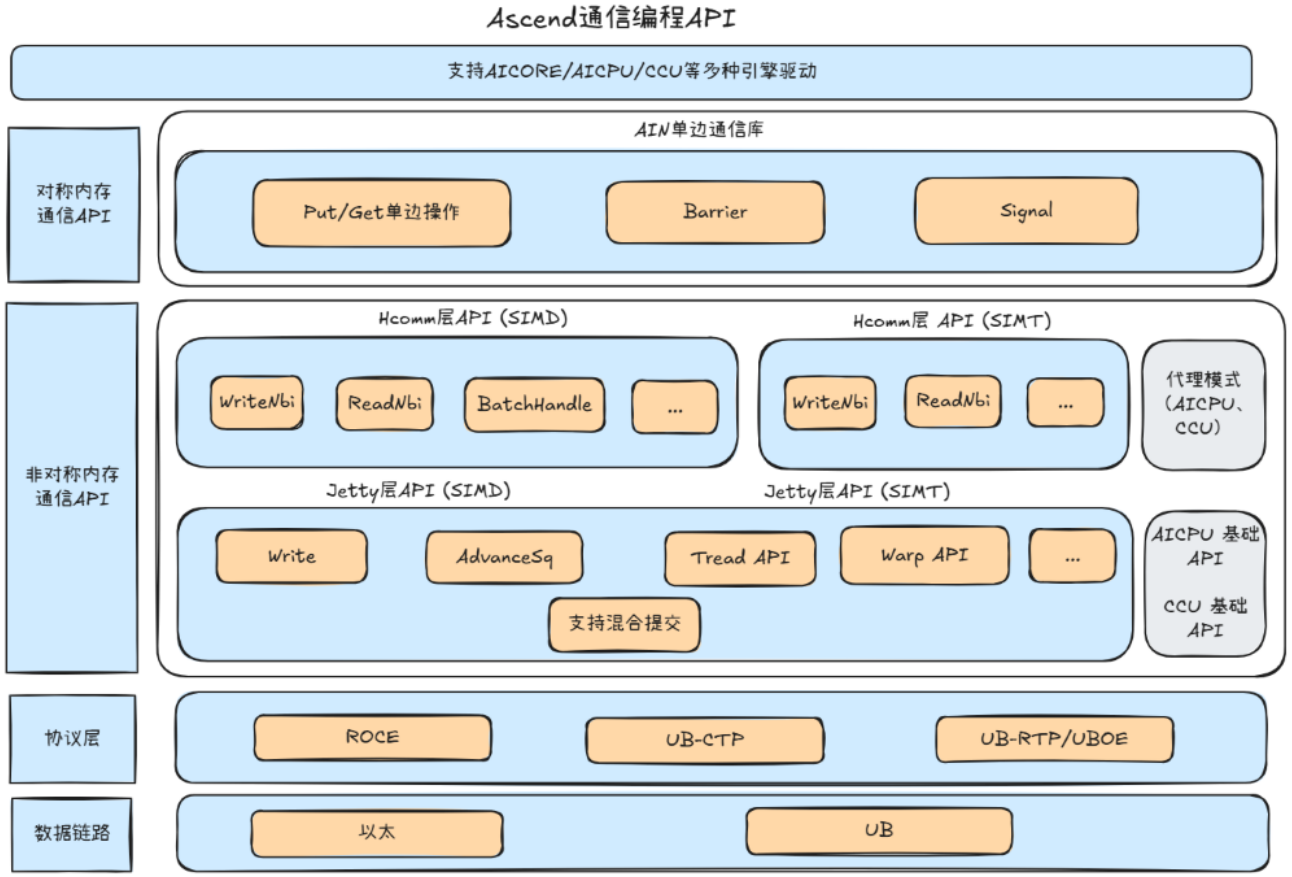
- **AICORE 直驱**：占用 Vector Core，发起路径最短
- **AICPU + TS 代理**：不占 AIV，通信与 kernel 解耦
- **CCU**：集合通信专用硬件，片上 buffer 减少 GM 往返
- 协议：**UB**（URMA / LD-ST，走灵衢）与 **RoCE**（以太网 RDMA）

DeepEP-Ascend 的落点：EPBuffer 基于 **Jetty 层 URMA 原语 + AIV 直驱**——kernel 内 warp 直接拼装 SQE、写 GM 上的 SQ、敲门铃，把延迟压到最低。

对照：CANN 官方参考实现（ElasticBuffer 的 moe_ep_dispatch / moe_ep_combine）用 **Hcomm 层 API**——WriteWithNotifyNbi 交换计数、MakeBatchHandle + WriteNbi + BatchCommit 批量写、Drain 等完成。

来源：CANN《AscendC 高性能自定义通信编程参考实践》（cann-recipes-infer · deepseek_v4.1_asc_comm_tech_report）。

asc-comm 整体架构：通信编程 API 全景



- **引擎层**：同一套 API 支持 AICORE 直驱 / AICPU+TS 代理 / CCU 三种引擎驱动
- **对称内存**：AIN 单边通信库——Put / Get、Barrier、Signal，按目标成员 + 窗口 + 偏移寻址
- **非对称内存**：Hcomm 层（SIMD / SIMT 双界面：WriteNbi、ReadNbi、BatchHandle）与 Jetty 层 URMA 原语（Write、AdvanceSq、Tread / Warp API，支持混合提交）；右侧挂代理模式（AICPU、CCU）及两类基础 API
- **协议层**：RoCE、UB-CTP、UB-RTP / UBOE
- **数据链路**：以太网与 UB（灵衢总线）

图来源：CANN《AscendC 高性能自定义通信编程参考实践》「自定义通信编程 API 全景 · 通信编程架构」（cann-recipes-infer · deepseek_v4.1_asc_comm_tech_report）。

Dispatch 整体结构：一个 kernel，两个阶段

阶段一 dispatch_impl：路由 + 发送

① 每 AIV 负责一段连续 token，512 线程 SIMT persistent worker

② 路由分析：expert/rank 直方图（UB 原子加）+ 按 rank 去重

③ 跨 AIV 前缀和 → 每个 (rank, token) 分到对端接收槽位

④ warp 协作拼装 SQE 写 GM 上的 SQ，ring doorbell 发出 URMA WRITE

⑤ 本地 rank 的 token 走 GM→UB→GM 多级流水直拷

⑥ metadata（topk_idx/weights/SF）先行打包，经 UB 流水写 metadata_send_buffer

阶段二 copy_epilogue：接收重排

① barrier 确认所有 URMA 写已落地

② 向量直方图指令统计本地 expert 计数（256/512 bin 硬件指令）

③ 跨 AIV 用 atomic-add 广播技巧合并计数，cumsum 得每专家起始槽

④ 按 expert_alignment=128 对齐做 zero padding

⑤ 把接收缓冲按 expanded layout 重排拷到 recv_x，直接可喂 grouped GEMM

两阶段拆分的动机：阶段一让通信尽早发出（通信是瓶颈）；阶段二的接收重排可延迟执行（defer_epilogue）与共享专家计算重叠——把"必须等通信完成"的工作压到最小。两阶段之间用 GM 上的计数交换协议交接。

核心算法（一）：路由直方图与按 rank 去重

路由阶段的输入、输出与约束

•输入：topk_idx（每 token top-k 个专家 id），expert e 属于 rank [e / 每 rank专家数]

•要算出：**每 rank 接收多少 token**（定发送量）与**每 token 在对端的槽位**（定落点）

•约束：64 个 AIV 并行完成，且不能回 Host

做法：UB 原子加直方图 + warp 去重

•每个 AIV 在 UB 建私有 expert/rank 直方图，512 线程扫自己分片的 token，**UB 内原子加**统计

•一个 token 的 top-k 若多个专家落在**同一 rank**，只发一份：simt::deduplicate 按 rank 去重，warp 内 ballot 压缩

•去重结果同时决定该 token 占几个 SGE 槽

metadata 先行

•topk_idx、topk_weights、src_token_global_idx、FP8 SF 打包成定长 metadata 记录

•经 UB 多级流水（MTE2→MTE3 stage）写入 metadata_send_buffer，随 SQE 一起 URMA 到对端

•对端凭 metadata 还原每个槽位的源 token——combine 的路由信息全靠它，不再重算

核心算法（二）：跨 64 个 AIV 的前缀和与槽位分配

接收缓冲按 (目标 rank × 槽位) 组织。AIV i 给 rank r 发 $n_{i,r}$ 个 token，则 AIV i 在 rank r 段的起始偏移 = $\Sigma_{j<i}\ n_{j,r}$——一个按 AIV 编号的**前缀和**，且要对每个 rank 各算一份。token 并行撒在 64 个 AIV 上，这个前缀和必须设备内完成。

三轮 warp 归约（GM 上的 local_rank_histogram）

轮 1 • 发布
各 AIV 把 rank 直方图累加进 GM 共享区，原子加打包状态字（值 + 到达数）

轮 2 • 前缀和
warp inclusive sum 扫 64 个 AIV 的计数，得每个 AIV 对每个 rank 的独占起始偏移

轮 3 • 分配
各 AIV 在段内按 warp lane 顺序给 token 编号落槽，SGE 目标地址随之确定

关键机制：打包状态字（全库复用频率最高的原语）

高 8 位 到达数	**低 24 位**：归约值（计数/直方图）

- kReductionIdentity = 1<<24，一次原子加同时推进值与到达计数——"等齐 64 个 AIV"和"求和"合并，无锁、无 barrier 指令
- 同一机制复用于 copy epilogue、combine、rank barrier、MegaMoE 进度同步；所有自旋带超时 trap，hang 变报错

核心算法（三）：SQE 拼装与 URMA 发送路径

SQE 结构（SQ 深度 32768，WQEBB=64 B）

48 B 头部：opcode = WRITE(0x3)、长度、目标 EID/token

6 × SGE 对（各 16 B）：token hidden 一段 + metadata 一段交替

末个 SQE 置位：odr=5 + fence + cqe → 强序与完成通知

warp 拼装：一次 warp 存储一条 SQE

- 32 lane 分工：lane 0–5 各写一个 SGE，其余 lane 填头部字段
- SQ 在 GM 上，kernel 写 SQ 即普通显存写，**提交零 Host 参与**
- 本地 rank 的 token 不走路径：GM→UB→GM 多级 MTE2/MTE3 流水直拷，与远程发送并行

提交与确认时序

① **advance_sq** 绕过 DCache 写 SQ head（MMIO 走 bypass，免 cache 一致性延迟）

② **ring_doorbell** 写 doorbell 寄存器 + DDR DSB 屏障（SQE 先于门铃可见）

③ **网卡执行 URMA WRITE** 数据直达对端 HBM，对端零参与

④ **poll_cq** CQE owner bit 翻位即完成，CQ 环形复用免清零

批量门铃摊薄 MMIO：dispatch 按 rank 聚一批 SQE 只敲一次门铃（combine 每 8 个远程任务一次）——高带宽的直接原因之一。

核心算法（四）：copy epilogue 与跨 rank 计数交换

copy epilogue：乱序到达 → GEMM 可吃的形状

•**向量直方图指令** asc_frequency_histogram_bin0/bin1（256/512 bin）一条指令扫完本地 expert 计数，比 SIMT 循环快一个量级

•**原子广播合并**：asc_set_atomic_add_int + 一次 ub2gm 写多份，各 AIV 拿到全量计数后 cumsum 出每专家起始槽

•**128 对齐 + zero padding**：下游是 DeepGEMM 的 MGroupedContiguous grouped GEMM，组起始要求 BLOCK_M 对齐且组内无垃圾数据——epilogue 直接产出合规缓冲，GEMM 侧零预处理

defer_epilogue：整段 epilogue 延迟到 .wait() 执行，与共享专家 GEMM 重叠
cached mode：解码时复用 handle 里的槽位与计数，跳过路由重建，压延迟

计数交换：接收方确认「收齐」

URMA 是单写语义，接收方不知道会来多少 token。发送方在数据之外再把**发送计数 URMA 写进对端计数区**：

① 数据 SQE → ② 计数 SQE（依赖 fence 强序，数据先到）→ ③ 接收方相位翻牌 barrier 等齐所有源 rank → ④ epilogue 启动

Host 侧同步与错误防护

•Host 需要计数时：kernel 经 pinned **host_workspace** 写出，CPU 自旋读打包状态字，免一次显存拷贝 API

•所有设备内自旋带**超时 trap**：超窗触发设备断言，集群 hang 变成带现场的错误

•计数区靠相位编码复用，每轮免清零——与 barrier 翻牌同源

Combine 整体结构：任务模型与两条数据通路

combine 是 dispatch 的逆过程：专家输出分散在各 rank，按**源 token × top-k 加权求和**送回。一个源 token 的 k 份结果散在 k 个 rank 上，**求和发生在哪、通信几次**直接决定带宽。路由信息全部复用 dispatch 的 handle 与 metadata，不重算。

调度结构

- **CombineTask = 8 个 token**：warp 按 lane 分工，一次处理 8 个源 token 的回收
- 每个 token 的 top-k 槽位用 **ballot / popc** 压缩成紧凑清单
- 每 8 个远程任务敲一次门铃；末尾补强序 NOP + CQE
- top-k 权重随 token 一起传回源 rank，最终加权在源侧 epilogue 完成——权重不走第二次通信

两条数据通路（按 num_slots 自动选择）

路径 A：单槽直通（num_slots == 1）

专家输出 → URMA WRITE → 源 rank 接收缓冲，零中间拷贝

路径 B：本地规约后发送（num_slots > 1）

k′ 份同 rank 结果 → RingReducer 加成 1 份（FP32）→ URMA WRITE，省 (k′−1) 倍跨 rank 流量

对偶设计：dispatch 的 metadata（槽位、源 token 索引、权重）在 combine 被完整复用——发送方留痕、回收方凭痕还原，**handle 即路由表。**反向传播时，输出梯度的反向 = 一次 dispatch，输入梯度的反向 = 一次 combine，四算子共享同一套 URMA 通道。

核心算法（一）：RingReducer，UB 上的环形规约器

同一源 token 的多份结果先在本地加成一份再发。这些份由不同 warp 在不同时刻取到——**多生产者、单结果**，且流水线不能 stall。RingReducer 用 UB 环形槽 + 三级管线互斥锁解决。

结构

- UB 上开若干**环形槽位**，每槽存一份 FP32 部分和
- 管线三段：**MTE2 读入** → **V 向量加** → **MTE3 写回**，段间 asc_lock/unlock 互斥，相邻 token 的不同段可重叠
- 新到的一份与槽内部分和向量加，凑齐 k′ 份的槽标记完成，交给发送路径
- 通信两端是 BF16/FP8，**规约全程 FP32** 保精度

互斥锁与原子加的取舍

- 向量加是**整条 UB 行**的操作，原子指令只覆盖标量/短向量，因此选锁
- 锁粒度 = 槽位 × 管线段，竞争概率低，实测规约吞吐不构成瓶颈
- 代价：combine 比 dispatch 多一次 UB 往返——两者带宽差的主要来源之一

权衡一览：本地规约把跨 rank 流量降到 1/k′，代价是 UB 占用与一次额外读写；k′=1 时自动走路径 A 绕过。
top-k 越小、专家分布越分散，combine 越接近纯转发。

核心算法（二）：master slot 跨 rank 累加（reduce epilogue）

源 rank 收到来自 k 个 rank 的 k 份"已本地规约"的结果，combine_reduce_epilogue_impl 把它们按 top-k 权重加成最终输出。核心是 **master slot** 机制。

master slot：每个 dst rank 只留一个槽

- 一个 token 的 top-k 清单里，对同一 dst rank 只保留**最后一个槽位**作为该 rank 部分和的落点
- 各 rank 部分和到达后按槽位清单跨 rank 累加，同时乘上随路带回的 top-k 权重
- 权重在传输中打包压缩，接收侧用 **OR 指令恢复**，不解码整条 metadata

细节：bias 与 deferred 执行

- 支持**最多 2 路 bias** 在累加阶段一并加入，省一次 element-wise kernel
- 与 dispatch 对称：通信先发，reduce epilogue 延迟到 .wait()，可与反向共享专家梯度计算重叠
- cached mode 复用 dispatch 缓存的槽位与计数

dispatch × combine 全景：dispatch 留痕（metadata），combine 凭痕还原；前向 combine 与反向 dispatch 互为镜像。
这套对偶设计让两次 all-to-all 共享通道、协议与同步原语，是 EPBuffer 最简洁的部分。

EPBuffer 性能设计：local 处理 + URMA 通信 + Epilogue 三段流水

极致的 local 处理性能与通信性能，**URMA 背景通信释放 AIV**，dual-pipe 并行提升端到端性能

Dispatch



Combine



· 三段按块流水、互相重叠；URMA 由网卡引擎代办，通信期间 AIV 空出做 local 处理（dual-pipe）

· combine 先做本地规约再回传，通信量随本卡专家数成比例下降

关键性能技术设计

local 性能优化

· SIMT 统计与去重

· 多 jetty 多核并发组装通信报文

· 批量 WQE 组装提升组装报文效率

· Epilogue 多 buffer 流水用满访存带宽

通信性能优化

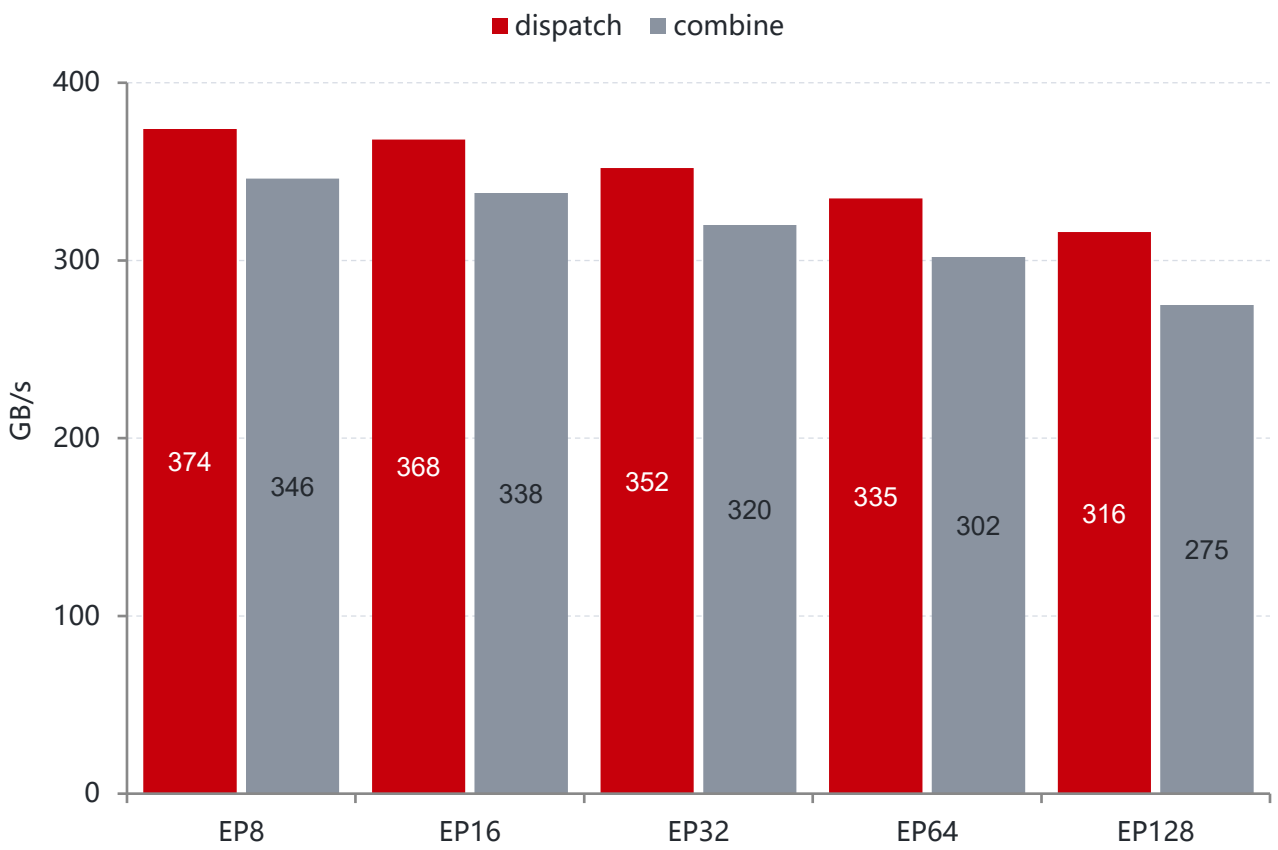
· 多 SGE 拼接多段 token 与 metadata，提高通信 payload 大小

· 多 jetty 并发用满 scale-up 多口带宽

· combine 支持 local reduce，减少通信数据量

性能表现：通信带宽达到硬件理论的 90% 以上

DeepEP-Ascend 整体性能解读



EP8/EP128 为 README 实测区间均值，EP16–EP64 为区间中值示意；测试环境 Ascend 950DT + CANN 9.2.0。

解读

- **≤EP32 达物理带宽 90–95%**：超节点内 UB 直通，kernel 开销被流水掩盖
- **EP128 降至 ~316**：跨超节点走 Clos 网，属拓扑物理约束而非实现损耗
- **combine 系统性低 8%–12%**：RingReducer 多一次 UB 往返、URMA 与 HBM 读争用、部分和存在"凑齐才发"的串行依赖
- 优化空间：EP>64 拓扑感知路由、减少 combine 的 UB 往返

03

DeepGEMM-Ascend 仓库

总体功能概览（算子接口版图）

稠密 GEMM（整体结构 + 核心算法）· MegaMoE 算子 · 性能解读

DeepGEMM-Ascend 总体功能概览

大模型算子库，覆盖**低精度训练、推理与专用融合计算**；以下按算子接口族梳理能力版图

算子接口	覆盖场景	关键算子的核心技术
矩阵乘 GEMM：bf16_gemm_{nt,nn,tn,tt}、fp8_fp4_gemm_*、fp8_gemm_* 兼容别名	线性层、FFN 训练与推理：BF16、FP8×FP8、FP4×FP4、FP8×FP4	原生 MAD / MX 指令与 NZ 分形布局；FP8×FP4 路径在 AIV 查表解包 FP4；L1/L0 多级缓冲，重叠数据搬运与计算
MoE 分组 GEMM：m_grouped_*_gemm_*_contiguous、k_grouped_*_gemm_*_contiguous	M 分组：专家前向、输入梯度；K 分组：专家权重梯度累加	前缀和连续分组布局与常驻调度；按专家边界处理尾块，复用 GEMM 流水
张量收缩：einsum / fp8_einsum	按 batch / head 的特征投影；限定三类表达式，覆盖反向	维度置换映射到 Batched GEMM；复用 BF16 / FP8 分块与流水实现
注意力 logits：fp8_fp4_mqa_logits、fp8_fp4_paged_mqa_logits、bf16_sparse_mqa_logits_bwd	注意力索引打分：Prefill、分页 KV cache 解码、稀疏索引反向梯度	融合 QK^T、ReLU 与跨 head 加权归约；Split-KV 调度，分页读取与尺度预取；稀疏反向按索引聚集 K 并归约梯度
HC prenorm 融合：tf32_hc_prenorm_gemm、tf32_hc_prenorm_bwd_gemm	HyperConnection / mHC 训练：前向投影与行平方和、输入梯度、权重梯度	TileLang 前向融合 cast、平方和、TF32 GEMM；K-split 在 kernel 内归约，复用 B / L0C；Cube / Vector 协同，反向融合两次 GEMM
MegaMoe 融合：mega_moe_fp8_fp4	Dispatch + Moe FFN + Combine + 共享 FFN 的大融合	基于 ubmem 通信，Dispatch→L1→L2→Combine 按块流水并行，掩盖通信开销

整体结构：__mix__(1,2) 混合 kernel 的角色分工

AIC（Cube 核）

只管 MAD：GM→L1→L0A/L0B 双缓冲喂数，asc_mmad / asc_mmad_mx 打到 L0C

块尾把 L0C FIX 到 UB 或直接 nz2nd 写回 GM

AIV0（Vector 核）

协程角色：FP4→FP8 权重反量化（LUT 查表流水）、SF 预取与重排

处理好的 B 经 ub2l1 喂进 AIC 的 L1B

AIV1（Vector 核）

epilogue 角色：drain UB 中的 L0C 结果，做 ScaleByAlpha / 动态量化写回 GM

__mix__(1,2)：每个 AIC 配 2 个 AIV。persistent kernel + Scheduler 分块，整个 GEMM 只启动一次；三类角色由编译期模板参数决定，不是运行时分支。AIC 与 AIV 只经 L1 / UB + 事件交接——这是与 CUDA warp specialization 的本质区别。

主数据通路（FP8 MX GEMM）



•SF 走独立通路（L1 SF stage 环 → L0_MX），不与 A/B 争 L0A/L0B 容量

•输出两条路：Identity epilogue 时 L0C→GM 直接 nz2nd **direct store**；否则 FIX 到 UB 由 AIV1 做 ScaleByAlpha / QuantizeToFP8（动态 UE8M0 + amax 归一 + 输出 SF）

核心算法（一）：布局约束链与多级双缓冲流水

MAD 约束与 NZ 分形布局

•**MAD_K 必须 64 的倍数**；L0A/L0B 数据必须是 **NZ 分形布局**（FRAC_MN=16，分形行 32 B）

•GM 行主序 → 进 L1 时拷贝硬件顺带 **nd2nz**，不占 AIV 算力；MN-major 源（如 SF）L1→L0 再转一次

•约束链：**L0C 256 KB** 卡 BLOCK_M×BLOCK_N → MAD_M/N 从 {1,1}/{1,2}/{2,1} 分裂里选 → MAD_K 由 L0A/B 容量与带宽对齐决定

•尾块 fill_tail_block 补零，保持 MAD 指令形状恒定

四级流水：每一级双缓冲、可调

流水级	缓冲策略
GM → L1（nd2nz）	L1 stage 环；A/B 跨 block 复用（stage 指针回绕）
L1 → L0A/L0B	L0 双缓冲：MAD 当前块时预取下一块
SF：GM→L1→L0_MX	sf_k_blocks 分块的独立 stage 环
L0C → 输出	epilogue stage 与 MAD 重叠

级间同步：事件对，不是 barrier

•MTE1 / MTE2 / MAD / FIX 各管线类型两两 **set_flag / wait_flag**，只等依赖的那一级，不做全核 barrier

•所有 stage 数由 Heuristics 搜索决定并编码为编译期常量；首拍填充是唯一的"等数据"窗口

98%+ 峰值兑现的来源：MAD 单元几乎不停——所有搬运提前一级完成，等待时间被压在首拍填充里。

核心算法（二）：MX 缩放因子的打包布局与专用通路

UE8M0 SF 的存储约定

•FP8/FP4 的 K 维每 **32个元素共享一个 UE8M0 缩放因子**（8 位纯指数），MAD 硬件边乘边还原动态范围

•Ascend 要求：每一对相邻 K 块的 SF 打包成一个 **int16**，整体按 **MN-major** 排布

SF 记录：K 块 i 与 i+1 → 一个 int16（两个 UE8M0 字节）

排列：MN 维为主序 → MAD 沿 K 前进时 SF 访问连续

SF 的独立流水线

•SF 走**自己的 L1 stage 环**（sf_k_blocks 分块）与 **L0_MX** 拷贝，不占 L0A/L0B

•asc_mmad_mx 一条指令同时吃 A/B 分形与 SF，无需软件缩放

•SF 支持**跨 block 复用**（resident SF）：同列板的 SF 只搬一次

•尾块 SF 随 fill_tail_block 补默认值，保持指令形状

结果：FP8 MX 与 BF16 的 MAD 管线利用率几乎相同——SF 开销被完全隐藏在独立通路里。

•同一 kernel 模板兼容 FP4：差别只在 L0 装载密度（K 维有效元素翻倍），SF 通路不变

•SF 的 stage 数与 resident 策略纳入 Heuristics 同一搜索空间，与 A/B 流水联合优化

核心算法（三）：FP4 权重的 LUT 反量化协程

FP8×FP4 反量化 GEMM 的目标：**存储减半（FP4 权重），算力照吃 FP8。**MAD 硬件不认 FP4×FP8 混合输入，解法是 AIV 协程在搬运途中把 FP4（E2M1）反量化成 FP8（E4M3），MAD 看到的永远是 FP8×FP8。

AIV0 的反量化流水（协程式）

① 建查找表
vf_build_lut：16 项 E2M1→E4M3 映射进向量寄存器（只建一次）

➡

② 寄存器查表
vf_deq_nd2nz：asc_gather 按 FP4 值当索引取回 FP8 字节

➡

③ 还原与布局
intlv 交织还原字节序，按 NZ 分形写 UB

➡

④ 喂进 L1B
asc_copy_ub2l1：UB→L1B，AIC 照常 MAD

两个工程细节与收益核算

- **LUT 优于换算**：E2M1→E4M3 是 4 bit→8 bit 纯映射，16 项查找表一条 gather 完成；换算需多条位操作，向量吞吐差数倍
- **dual AIV 双路解码**：权重大时单个 AIV0 解码跟不上 MAD 消耗，Heuristics 判定启用两个 AIV 交替反量化，吞吐翻倍
- **收益核算**：权重 GM 流量减半，解码开销落在空闲 AIV 上，MAD 吞吐不变——dense FP4 实测 1701–1730 TFLOPS，达峰值 98.3%–99.8%

核心算法（四）：块调度 swizzle 与 Heuristics 自动调优

Scheduler：persistent + L2 友好 swizzle

•block_idx = iter × numCores + core_id：32 个 AIC 常驻循环领块，整个 GEMM 只启动一次

•**GROUP_M=4 分条 + 条内 XOR 换序**：同时执行的块尽量共享 A 行板 / B 列板，L2 命中最大化

•**snake（Z 序）**：相邻条反向扫描，块间跳转距离最小；尾部不足一条回退行主序

•四种分组模式共用模板：**Normal / MGroupedContiguous（psum 前缀和定位组、BLOCK_M 对齐组起始）/ KGrouped（K 按组拼接、D 按组）/ Batched**，差别只在块索引映射函数

Heuristics：显式代价模型（722 行）

•搜索顺序：**L0C 256 KB 筛 BLOCK_M/N** → MAD_M/N 三种分裂 → MAD_K 与 L1/L0/SF/epilogue stage 数 → L2 cache hint 与 refine（direct store / resident / dual AIV）→ 再扫 BLOCK_K

•代价项：GM/L2 流量（sector 128 B 对齐放大）、首拍填充、L0 拷贝放大、tail 波次折扣、issue/turnover 开销

•硬件参数全部在 **HWDesc 表**：换芯片只换表；搜索结果编码进 JIT 常量，运行时零开销

调优与编译是同一个动作：Heuristics 的输出直接成为 DeepJIT 的编译常量，kernel 内零"选路"分支；
这里的代价模型完全显式、可读、可改。

MegaMoE 整体结构：一个 kernel 装下整个 MoE 层

MegaMoE（1221 行）把 **EP dispatch + 两个 grouped GEMM + SwiGLU 激活 + combine** 融合进单个 kernel（EP8、top-6 + 1 共享专家）：消掉算子间所有 GM 往返与 kernel 边界，让通信和计算在 kernel 内部自由重叠、互相掩盖耗时。

三角色分工（编译期决定）

AIC：两个 Linear 的 GEMM

shared + routed 两路，FP8 激活 × FP4 权重，复用稠密 GEMM 的整条 MAD 流水

AIV0：反量化 + dispatch 元数据

FP4→FP8 NZ 权重解码喂 L1B；SIMT 建 expert 直方图 → grid_sync → 前缀和 → 远程写对端 metadata / recv_count

AIV1：SwiGLU 量化 + 通信搬运

vf_swiglu_quant 融合 epilogue；dispatch pull（直读对端 GM）与 combine push（按 src_metadata 写回源 rank），交错调度

按块流水：1 AIC + 2 AIV 并行分工



图来源：社区直播材料《DeepEP DeepGemm 开源库深度解读》

对称 workspace 与进度同步

- 交换数据放**对称 GM workspace**，布局为编译期常量，远端地址免协商
- 进度用 **4 阶段原子计数器** + rank barrier（epoch 翻牌）
- ring buffer 复用输入/输出行；combine reduce 与大消息通信重叠（>32 MiB 阈值）

MegaMoE 核心算法：pull / push 通信与细粒度通算重叠

dispatch pull + combine push

•与 EPBuffer 的"发送方 push"相反：**接收方按路由 index 主动 pull**，直读对端 GM，省一次计数交接；同一份输入供多专家读取，省展开副本的缓冲

•源行按 **coprime stride 置乱**：互质步长打乱访问序，避免多 rank 同读一区造成 HBM 热点

•combine push 对称：按 src_token_metadata 写回源 rank 输出行

vf_swiglu_quant：七步合一的 epilogue

clamp → sigmoid 门 → 乘 up 投影 → 乘路由权重 → amax 求 UE8M0 → FP8 量化 → 输出 SF。

传统实现是 3–4 个独立 kernel、每步一轮 GM 读写；这里全部在 UB 内完成，**中间结果不出片**，输出直接是 Linear2 可吃的 FP8+SF。GMM1 分块交错排列 gate/up，算完一块即可激活。

调度说明：按块 Dispatch 拉取完成即推动下游 GEMM；**Linear1 / Linear2 交错调度**最大程度用满 AIC；多 buffer + L1 多级预取提升 MAC 利用率；GMM 环形工作区减少内存开销。

Wave 调度：通信被计算掩盖

•专家输入按 ≤256 行分组、若干组一个 Wave；tile 就绪即算，不等整个专家输入

•**前一 Wave 的 combine 与后一 Wave 的 GMM1 重叠**；AIV1 交错处理 dispatch 与 combine

•带宽竞争控制：启动先搬首批输入，计算期间限流并发通信，收尾加速传剩余结果

•kOverlapReduce：消息 >32 MiB 时规约与传输分段重叠

实测

指标（950DT 环境）	实测
计算吞吐（384 专家/16k token）	846 TFLOPS + 通信 103–135 GB/s
Decode 加速比（vs 分离算子 / vs 基线）	**1.27–1.52× / 2.5×+**
Prefill 加速比（vs 分离算子 / vs 基线）	**1.59–1.90× / 1.5×+**
Prefill Cube 利用率	**93% 以上**

来源：README 与 asc-comm 技术报告（vs 分离算子）；vs 基线与 Cube 利用率来自社区直播材料。

DeepGEMM-Ascend 整体性能解读

算子	实测	峰值占比 / 说明
dense FP4 GEMM	**1701 – 1730 TFLOPS**	硬件峰值 98.3% – 99.8%
dense BF16 GEMM	431 – 432 TFLOPS	≈99%
M-grouped GEMM（MoE 前向）	818 – 862 TFLOPS	分组形状碎、负载不均下仍高位
MQA logits	—	FIX 管线利用率 99%，贴近搬运上限
HC prenorm	3463 GB/s	逼近 HBM 写带宽上限

数据来源：DeepGEMM-Ascend README 性能章节（Ascend 950DT + CANN 9.2.0，DeepJIT 编译）。

解读

- **dense GEMM 已贴近硬件天花板**：工程价值在于把峰值兑现到 grouped / 低精度 / 融合场景，M-grouped 仍保持 818–862 TFLOPS 即为证据
- **性能来源可拆解**：多级双缓冲流水（等数据时间压在首拍）+ NZ/SF 布局约定 + AIV 协程卸载反量化与量化 + 编译期特化（Heuristics × DeepJIT）
- 与 DeepEP 的接口约定（128 对齐 expanded layout、MN-major SF）让两个库零拷贝衔接；MegaMoE 是把两者融成单 kernel 的极限形态

总结

1 • 硬件　Atlas 950 SuperPoD 用灵衢 UB 把 8192 卡做成一颗"大芯片"（统一编址、URMA 直访）；950DT 单芯 32 AIC + 64 AIV、HBM 3500 GB/s、L2 128 MB——两个库的所有设计都能回溯到这组数字。

2 • 通信　DeepEP-Ascend 站在 asc-comm 的 Jetty/URMA 层上，用"每 token 一条 WQE"的直通数据面加全设备内同步协议（打包状态字、翻牌 barrier），EP≤32 打满物理带宽 90–95%。

3 • 计算　DeepGEMM-Ascend 用一套混合 kernel 模板 + 显式代价模型把 dense FP4 GEMM 兑现到峰值 98%+；MegaMoE 进一步把通信与计算融进一个 kernel，Decode 提速 1.27–1.52×。

来源：两仓库源码与 README、昇腾 950 架构白皮书、CANN asc-comm 技术报告、社区直播材料。谢谢。

Thank you.

社区愿景：打造开放易用、技术领先的 AI 算力新生态

社区使命：使能开发者基于 CANN 社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的 CANN 生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann